Heart Sound Classification from Digital Stethoscope Recordings

Prepared by Mourad Mohamed Gad

Project report - August 2026

Project overview

This project uses recorded heart sounds to classify a recording as normal or abnormal. When the model is not confident enough, it returns an uncertain result instead of forcing a prediction.

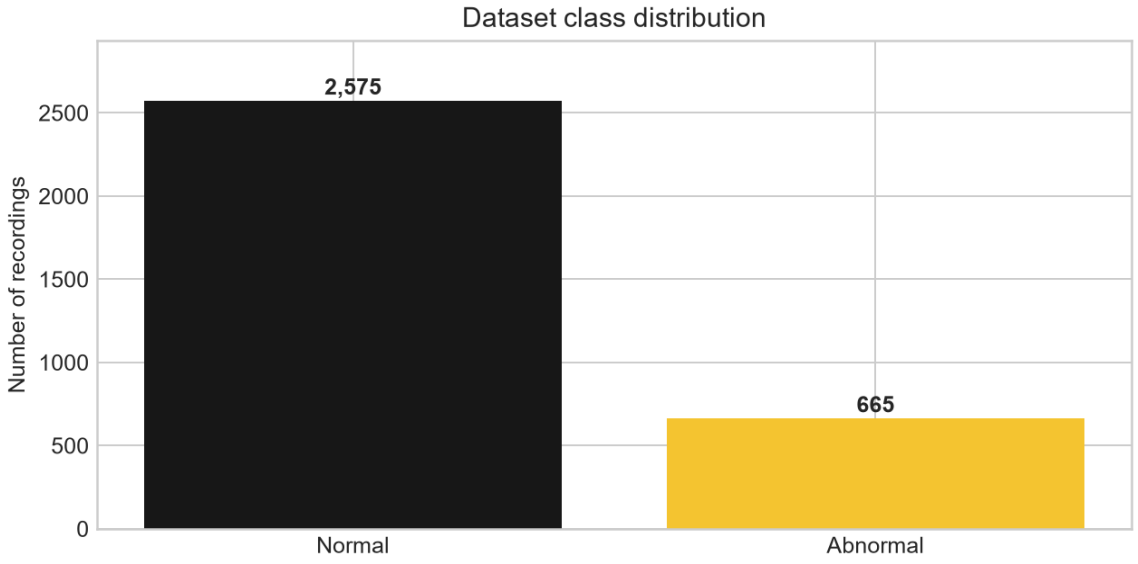
Main test results

Result type	Precision	Recall	Specificity	Coverage
Binary result	79.3%	92.9%	93.8%	100.0%
Final three-way result	92.1%	97.2%	99.1%	78.6%

Why two result types are shown

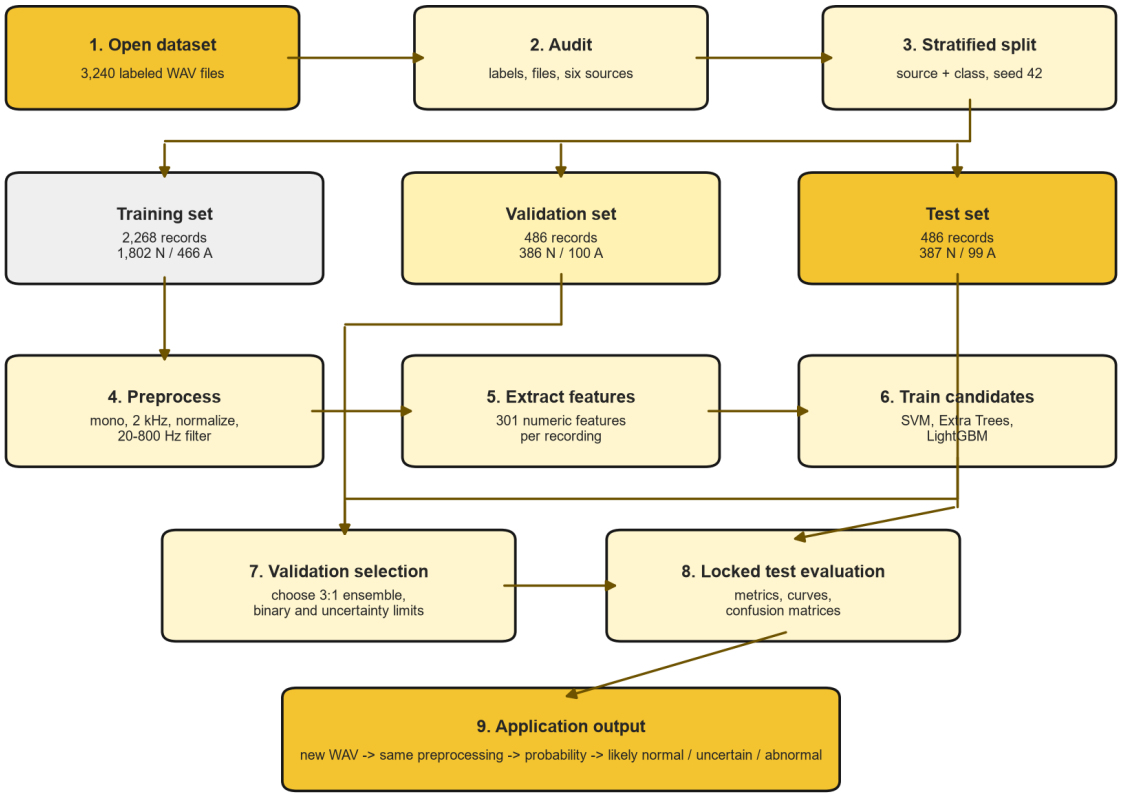
Result type	What it means
Binary result	The model must choose normal or abnormal for every recording. This is useful for comparing the model with a standard two-class test, but it can force a choice on difficult recordings.
Final three-way result	The model can answer likely normal, abnormal, or uncertain. The uncertain result is used when the probability is between the two confidence limits. This is the version used by the application.

The final result is calculated only for recordings that received a clear normal or abnormal prediction. The model gave a clear answer for 78.6% of the test recordings. The other 21.4% were marked uncertain.



Complete data journey

The diagram follows the data from the downloaded WAV files to the final application result. Training, validation, and testing have different jobs and are kept as separate parts of the workflow.



Training path

The training recordings are preprocessed, converted into 301 numeric features, and used to fit five candidate models. The training labels are visible to those models.

Validation path

The validation recordings are not used to fit the candidate model parameters. They are used to compare candidates, choose the ensemble weights, and choose the probability limits.

Test and application paths

The test set measures the selected system. In the application, a new WAV file follows the same preprocessing and feature extraction before the saved models calculate its abnormality probability.

The dataset

The project uses the public PhysioNet/Computing in Cardiology Challenge 2016 heart sound dataset. The recordings were collected by several research groups in clinical and non-clinical environments. They include both children and adults and contain real recording noise such as breathing, speech, movement, and stethoscope contact noise.

Where the data came from

Dataset page: https://physionet.org/content/challenge-2016/1.0.0/
Dataset license: Open Data Commons Attribution License v1.0
Downloaded file used in this project: training.zip

Data used

Item	Value
Total labeled recordings	3,240
Normal recordings	2,575 (79.5%)
Abnormal recordings	665 (20.5%)
Source databases	6
Audio format	WAV
Standard sample rate	2,000 Hz
Training / validation / test	70% / 15% / 15%

What the labels mean

Normal means that the recording was labeled as a normal heart sound. Abnormal means that the patient had a confirmed cardiac abnormality. The dataset does not provide a separate disease name for every recording, so this project predicts normal or abnormal rather than a specific disease.

How the recordings were prepared

Each WAV file was converted to one audio channel and resampled to 2,000 Hz. A 20-800 Hz filter was applied to keep the main heart-sound frequency range. The audio level was normalized. The program then calculated MFCC, spectral, energy, timing, frequency-band, and wavelet features from each recording.

All 3,240 recordings were processed successfully. The abnormal class was smaller, so class balancing was used during training.

Data split and preprocessing

The split was produced with random seed 42. Stratification used both the source database and the class label so that each split retained a similar source and class mixture.

Split	Total	Normal	Abnormal	Purpose
Training	2,268	1,802	466	Fit model parameters
Validation	486	386	100	Choose models and thresholds
Test	486	387	99	Measure selected system

Preprocessing steps

Step	Exact operation	Reason
1. Load	Read WAV and convert to floating-point audio	Create a consistent numeric signal
2. Mono	Combine channels into one channel	The model expects one PCG signal
3. Resample	Convert every file to 2,000 samples per second	Give every recording the same time scale
4. Minimum length	Pad recordings shorter than one second with zeros	Avoid invalid short inputs
5. Center	Replace invalid values and subtract the median	Remove the DC offset
6. Normalize	Divide by the 99.5th percentile absolute amplitude and clip to -1 to 1	Reduce recording-level gain differences
7. Filter	Fourth-order Butterworth band-pass from 20 to 800 Hz	Keep the main PCG range and reduce unrelated frequencies
8. Frame analysis	256-sample FFT window with 64-sample hop	Measure how the signal changes over time

No audio augmentation, synthetic samples, or deep-learning image input was used. The spectrograms in this report are visual explanations; the model itself receives the 301 numeric features described next.

Feature extraction

Feature extraction converts a variable-length audio recording into one fixed row of 301 numbers. This allows fast CPU models to process recordings of different durations.

Feature family	Calculation	Number
MFCC and delta MFCC	20 MFCCs and 20 first differences, each summarized five ways	200
Basic time/spectral	RMS, zero-crossing, centroid, bandwidth, roll-off, flatness; five summaries each	30
Spectral contrast	Five contrast bands with five temporal summaries	25
Recording/amplitude	Duration, clipping, silence, crest factor, and six amplitude percentiles	10
Physiological frequency	Five band powers, spectral entropy, envelope periodicity, heart-rate proxy	8
Wavelet	Seven db4 coefficient groups; energy, standard deviation, skew, and kurtosis	28
Total	One fixed feature vector for each recording	301

Temporal summaries

For each frame-based feature, the code calculates the mean, standard deviation, median, 10th percentile, and 90th percentile across the complete recording. These summaries keep information about the typical value, spread, and low/high behavior without requiring every recording to have the same duration.

MFCC and spectral features

MFCCs describe the short-term spectral shape of the sound. Delta MFCCs describe how that shape changes. Spectral centroid, bandwidth, roll-off, flatness, and contrast describe where frequency energy is located and how concentrated it is.

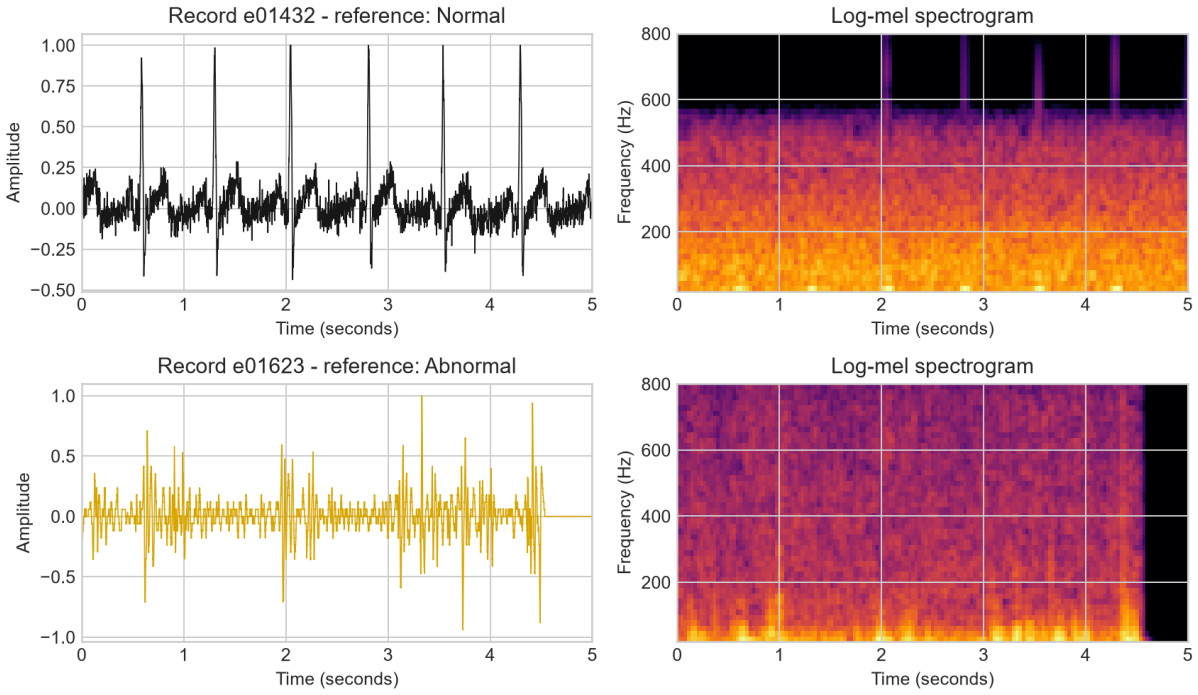
Periodicity and wavelets

Welch power spectral density is used for five bands: 20-60, 60-120, 120-200, 200-400, and 400-800 Hz. A Hilbert-envelope autocorrelation between 0.30 and 1.50 seconds provides periodicity and heart-rate proxy values. A six-level Daubechies-4 wavelet decomposition provides multi-scale energy and shape measurements.

The feature table was created for all 3,240 recordings with zero extraction failures. The measured extraction time was 79.33 seconds using six CPU workers on this computer.

What the heart-sound data looks like

The figure below shows two real recordings from the held-out test set. Only the first five seconds are displayed so the sound pattern is easy to see.



Waveform

The waveform shows how the recorded sound pressure changes with time. Repeating peaks usually follow the heart cycles. The waveform can also contain noise from breathing, movement, or stethoscope contact.

Spectrogram

The spectrogram shows the same recording using time on the horizontal axis and frequency on the vertical axis. Brighter areas contain more sound energy. This view helps show when sound energy appears and which frequencies contain it.

These plots are examples only. A waveform or spectrogram should not be diagnosed by visual appearance alone.

Models that were tested

Five CPU model candidates were trained on the same 2,268-record training split. Missing numeric values were filled with the median learned from the training data.

Candidate	Main settings	Validation F1	Validation AUROC
RBF SVM C=3	Robust scaling; class-balanced	0.762	0.933
RBF SVM C=10	Robust scaling; class-balanced	0.769	0.937
Extra Trees	900 trees; minimum leaf 1; 45% features; class-balanced	0.879	0.981
LightGBM balanced	700 boosting iterations; balanced classes	0.850	0.978
LightGBM moderate	700 iterations; abnormal weight 2.5	0.847	0.978

Why these models

SVM provides a nonlinear decision boundary after robust feature scaling. Extra Trees builds many randomized decision trees and averages them, which handles nonlinear feature interactions well. LightGBM builds trees sequentially so later trees correct errors made by earlier trees.

The final ensemble

Validation prediction blends were searched using small integer weights from 0 to 3. The selected blend was three parts Extra Trees and one part moderate LightGBM. Therefore, the final probability is 75% Extra Trees probability plus 25% LightGBM probability.

Final component	Exact settings
Extra Trees	900 trees; minimum samples per leaf 1; max_features 0.45; balanced class weights; all CPU cores; random seed 42
LightGBM	700 boosting iterations; learning rate 0.025; 15 leaves; minimum child samples 20; row fraction 0.85; feature fraction 0.75; L2 regularization 2.0; abnormal weight 2.5; six CPU workers; seed 42
Probability blend	(3 x Extra Trees probability + 1 x LightGBM probability) / 4

Training process and timing

This project does not use a neural network, so it does not have epochs. Epochs normally mean repeated full passes through training data in neural-network training. The tree and SVM methods use different stopping units.

Method	Equivalent training unit
Extra Trees	900 independent randomized trees are fitted once
LightGBM	700 boosting iterations; one new correction tree per iteration
SVM candidates	The numerical optimizer runs until its convergence condition

Training sequence

1. Fit the median imputer and each candidate using the 2,268 training recordings.
2. Calculate candidate probabilities for the 486 validation recordings.
3. Compare individual models and search integer probability-blend weights.
4. Choose a binary threshold from 0.05 to 0.95 in steps of 0.005 using validation F1 and balanced accuracy.
5. Save the selected models, feature order, blend weights, and threshold.
6. Search lower and upper validation limits for the three-way output.
7. Apply the saved system to the 486 test recordings.

Measured runtime

Stage	Measured time	Included work
Feature extraction	79.33 seconds	All 3,240 WAV files, 301 features, six CPU workers
Model training and selection	30.29 seconds	Five candidate fits, validation predictions, blend search, test prediction, and model save
Combined compute stage	109.62 seconds	Feature extraction plus model-selection training

The timing was measured on the local Windows environment used for this project with eight logical CPU threads and no accessible NVIDIA GPU. Download time, dependency installation, report creation, and human review are not included. Runtime will change on another computer.

Class imbalance

Only 20.5% of recordings were abnormal. Class-balanced weights were used for SVM and Extra Trees. The final LightGBM candidate assigned abnormal training examples 2.5 times the weight of normal examples. This helps the model avoid learning to predict the larger normal class too often.

How the saved application works

The Flask application runs locally and uses the saved model file. It does not retrain when a user uploads a recording.

Step	Application action
1. Upload	Accept a WAV file through the browser form
2. Temporary file	Save the upload temporarily so the audio library can read it
3. Preprocess	Apply the same mono conversion, 2 kHz resampling, normalization, and 20-800 Hz filter used in training
4. Features	Calculate the same 301 features and put them in the saved training-column order
5. Component probabilities	Extra Trees and LightGBM each calculate an abnormality probability
6. Ensemble	Combine probabilities as (3 x Extra Trees + 1 x LightGBM) / 4
7. Decision	Probability at or below 0.215: likely normal; 0.215-0.825: uncertain; at or above 0.825: abnormal
8. Display	Show the final label, abnormality probability, and confidence
9. Cleanup	Delete the temporary uploaded file

What is saved

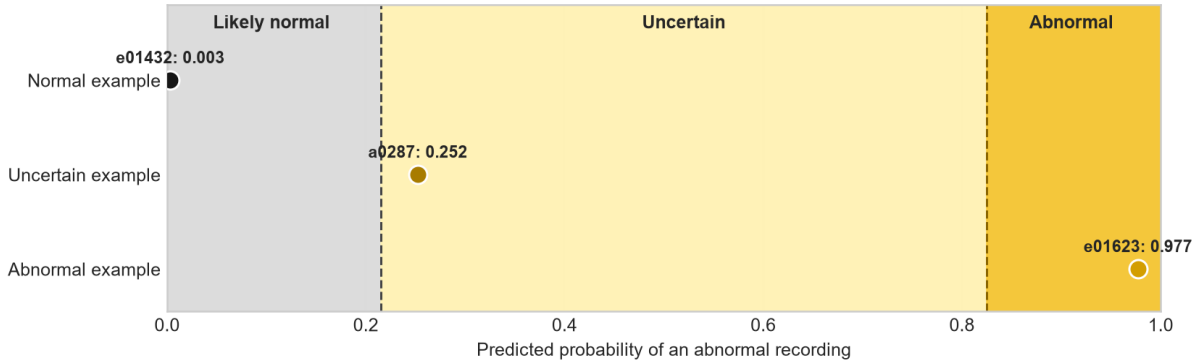
The model file stores the fitted Extra Trees model, fitted LightGBM model, their 3:1 weights, the binary threshold, the exact 301-feature column order, and a model version. The selective thresholds are stored in selective_metrics.json.

What the application does not do

It does not identify a named disease, replace an echocardiogram, provide treatment advice, or prove clinical safety. It produces a normal/uncertain/abnormal research output for a WAV heart-sound recording.

How the detection looked

The model converts the features from each recording into an abnormality probability. The two fixed limits then convert that probability into likely normal, uncertain, or abnormal.



Record	Reference label	Abnormality probability	Final output
e01432	Normal	0.003	Likely normal
a0287	Normal	0.252	Uncertain
e01623	Abnormal	0.977	Abnormal

Example 1: clear normal

Record e01432 received probability 0.003, which is below the normal limit of 0.215. The final output was likely normal.

Example 2: uncertain

Record a0287 received probability 0.252. This is above 0.215 but below 0.825, so the model did not force a normal or abnormal answer. The final output was uncertain.

Example 3: clear abnormal

Record e01623 received probability 0.977, which is above the abnormal limit of 0.825. The final output was abnormal.

The reference label is the dataset label. The final output is the result produced by the trained model.

Evaluation method

Every reported metric is calculated at recording level. Abnormal is treated as the positive class and normal as the negative class.

Term	Meaning
True positive (TP)	An abnormal recording predicted as abnormal
False positive (FP)	A normal recording predicted as abnormal
False negative (FN)	An abnormal recording predicted as normal
True negative (TN)	A normal recording predicted as normal

Metrics

Metric	How it is calculated	What it answers
Accuracy	(TP + TN) / all recordings	How often was the model correct overall?
Balanced accuracy	Average of abnormal recall and normal recall	How well did it handle both classes equally?
Precision	TP / (TP + FP)	When it said abnormal, how often was it right?
Recall / sensitivity	TP / (TP + FN)	How many abnormal recordings did it find?
Specificity	TN / (TN + FP)	How many normal recordings did it correctly reject?
F1 score	Harmonic mean of precision and recall	How balanced were precision and recall?
AUROC	Ranking performance over all binary thresholds	Did abnormal cases generally receive higher probabilities?
Average precision	Summary of the precision-recall curve	How good was positive detection across thresholds?
Coverage	Clear normal/abnormal outputs / all inputs	What fraction avoided an uncertain result?

Threshold selection

The binary threshold 0.295 was selected on validation predictions to maximize F1, with balanced accuracy as the next comparison. The final three-way search examined a lower limit from 0.05 to 0.40 and an upper limit from 0.45 to 0.90. It selected the highest validation coverage for which precision, recall, and specificity were each at least 99%, producing limits 0.215 and 0.825.

The train/validation/test split prevents direct fitting on the test rows. However, more than one project experiment was reviewed during development, so these figures should be treated as a development benchmark rather than a completely independent clinical validation. Patient identifiers are also not consistently available across all source databases.

The model and results

The final CPU model combines Extra Trees and LightGBM. Both are fast models for structured numerical features. The model produces a probability between 0 and 1, where a larger value means that an abnormal recording is more likely.

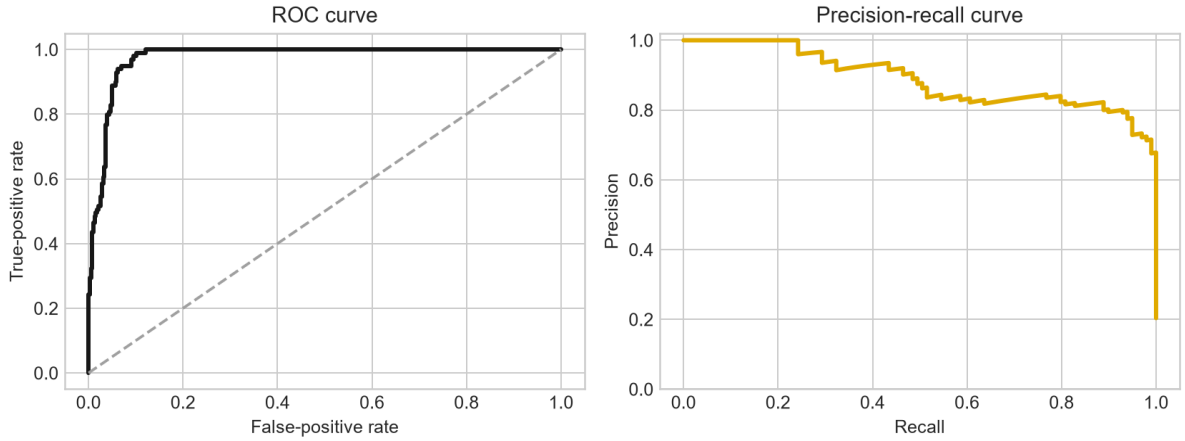
Binary test results

Metric	Result
Accuracy	93.6%
Balanced accuracy	93.4%
Precision	79.3%
Recall	92.9%
Specificity	93.8%
F1 score	85.6%
AUROC	0.975
Average precision	0.890

Binary confusion counts

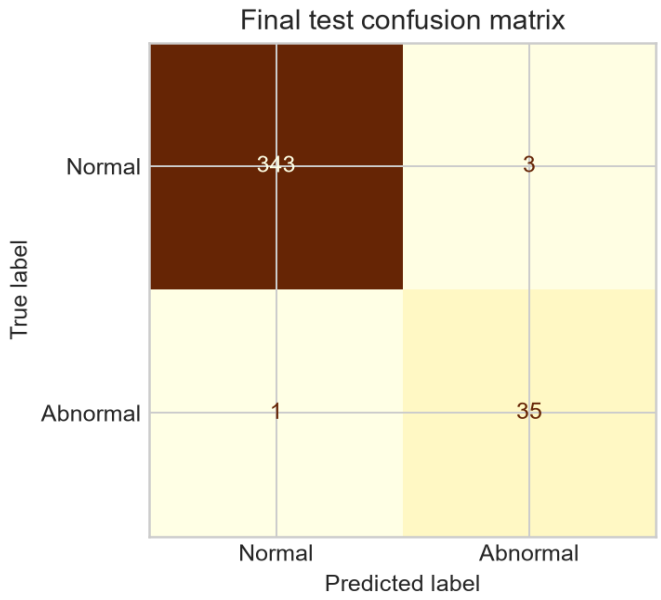
True normal	False abnormal	Missed abnormal	True abnormal
363	24	7	92

The binary model detected most abnormal recordings, but 24 normal recordings were predicted as abnormal. Because abnormal recordings are only 20.5% of the data, these false positives reduce precision more strongly than they reduce overall accuracy. For this reason, the final version uses an uncertain range.



Final three-way results

The final decision rules were fixed using the validation set. A probability at or below 0.215 is reported as likely normal. A probability at or above 0.825 is reported as abnormal. Values between these limits are reported as uncertain.



Measure	Test result
Clear predictions	382 of 486
Coverage	78.6%
Precision	92.1%
Recall	97.2%
Specificity	99.1%
Referral sensitivity	99.0%

Among the clear test predictions, 343 normal and 35 abnormal recordings were classified correctly. There were three false positives and one false negative. A further 104 recordings were marked uncertain.

These results describe this dataset and test split only. The project is a research demonstration and is not a medical diagnosis tool. Patient identifiers are not available consistently across all six sources, so the evaluation is performed at recording level.

References

1. Liu C, Springer D, Li Q, et al. An open access database for the evaluation of heart sound algorithms. Physiological Measurement. 2016;37(12):2181-2213. https://doi.org/10.1088/0967-3334/37/12/2181

2. Clifford GD, Liu C, Moody B, et al. Classification of normal/abnormal heart sound recordings: the PhysioNet/Computing in Cardiology Challenge 2016. Computing in Cardiology. 2016.

3. PhysioNet. Classification of Heart Sound Recordings, version 1.0.0. https://physionet.org/content/challenge-2016/1.0.0/